॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

B-Tech Project

ForgeFlow: An AI-Native End-to-End Software Development Lifecycle Automation Platform

Group Number: Group – 04

Raghav Sonchhatra – PRN 23070122172

Sanidhya Awasthi – PRN 23070122192

Faheemuddin Sayyed – PRN 23070122196

Guided By:

Dr. Payal Kadam

Professor

Computer Science

CONTENTS

Roadmap for Review 2

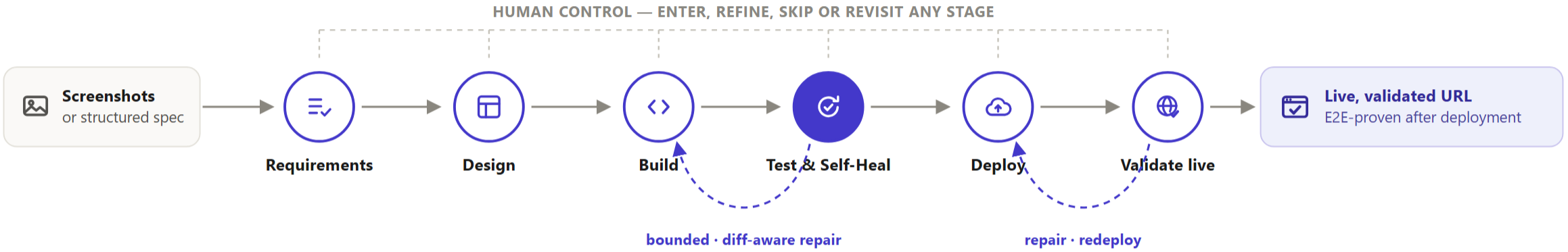
01	Introduction	03
02	Motivation	04–05
03	Status of Current Work	06
04	Research Gaps	07–09
05	Problem-to-Design Mapping as per the Gap Identified	10–11
06	System Architecture Diagram & Database Connectivity	12–13
07	Identified Tools / Technologies	14–15
08	Plan of Next Work	16
09	Conclusion	17
10	References	18

> “ Review 1 proved the pipeline exists end-to-end. Review 2 proves the gaps were closed on purpose — and where the design came from.

INTRODUCTION

ForgeFlow — An AI-Native, Human-in-the-Loop Platform for the Automated Web SDLC

Screenshots or a spec in → a tested, deployed website out.



PWA with an embedded browser IDE — Monaco editor and terminal built in; generated code stays visible and editable, not a black box.

No stage is mandatory. Enter anywhere, refine anything, skip anything, jump back at will — non-linearity is an invariant of the design.

Core technical contribution: a bounded, diff-aware self-healing repair loop — verification that terminates instead of thrashing.

Fixed generated stack — React + Express + MongoDB — gives higher first-pass success and makes rigorous verification tractable.

> Existing tools generate apps. ForgeFlow generates apps it can *prove* work — and then proves it again on the live URL.

The Adoption–Trust Gap Is Now the Central Problem in AI Software Engineering

Adoption is effectively universal

76% of developers now use — or plan to use — AI coding tools, **down from 84%** a year earlier [3]

41–46% of all new code is now AI-generated; Gartner projects **60% by end-2026** [29]

$4.7B → $12.3B prompt-to-app market, 2026 → 2027 (≈38% CAGR) [29]

63% of prompt-to-app users are **non-developers** — no ability to debug what is generated [29]

Trust has fallen even as adoption drops



46% **actively distrust** the accuracy of AI output [3]

66% cite "almost right, but not quite" as their single biggest frustration [3]

45% say debugging AI-generated code takes **longer than writing it themselves** [3]

MOTIVATION (2/2)

And the Damage Is Measurable — Failures Survive QA and Reach Production

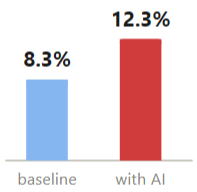
43%

of AI-generated changes still need manual debugging **in production — after passing QA and staging** [25]

26.8–30.2%

first-attempt success rate deploying two frontier models' own generated infrastructure — **deployment is a distinct failure mode from generation quality** [6]

88% still needed 2–3 redeploy cycles per fix [25]



Code duplication rose 8.3% → 12.3% across 211M lines analysed [26]; a 2026 mining study of AI-generated build code found the same class of maintainability & security smells [4]

1.7×

more issues in AI-generated code than human-written code [28]; **45% of it carries a security flaw** [27] — and **iterating "make it more secure" can weaken security further** [5]

2,500%

Gartner-projected **increase in software defects by 2028** from citizen-developer prompt-to-app without quality controls [29]

> Generation is solved. **Verification is not.** That gap is exactly where we built.

STATUS OF CURRENT WORK

First Working Prototype — Complete End-to-End Pipeline Implemented

Upload screenshots / spec → Capture requirements → Refine design → Generate app → Generate & run tests → Self-heal failures → Deploy → Validate live URL

53 / 53	≈41,000	1,491	✓ COMPLETE	10
PHASES SHIPPED · 12 EPICS	LINES OF PRODUCTION CODE	AUTOMATED TESTS PASSING	DEPLOYMENT-PHASE TESTING	BENCHMARK SPECS SEEDED

SHIPPED SUBSYSTEMS

- ✓ **Control plane** — FastAPI orchestrator, non-linear stage machine, conductor
- ✓ **PWA frontend** — Workspace, stage navigator, conversational refinement
- ✓ **Web IDE** — Monaco editor, xterm.js terminal, live code streaming
- ✓ **Execution** — Isolated per-project Docker sandboxes, live preview
- ✓ **Design stage** — Pluggable provider — Stitch MCP default, Figma alternative
- ✓ **Build stage** — Agentic codegen on the fixed React / Express / MongoDB stack
- ✓ **Requirements → tests** — Structured guided capture → unit + Playwright E2E suites
- CORE CONTRIBUTION — ✓ **Self-healing repair loop** — Diff-aware, bounded, escalating to a human on stall
- ✓ **Deploy** — Infra analyzer + dual-mode credentials (managed / BYO) — full app on Vercel
- TESTED THIS CYCLE — ✓ **Live validation** — E2E re-run against the deployed URL + repair path — deployment-phase testing complete
- ✓ **Data** — Per-project database isolation + in-app CRUD browser
- ✓ **Evaluation harness** — Headless pipeline runner + metrics collection

✓ **New since Review 1 — deployment-phase testing is complete:** the deployed URL is now re-run against the generated Playwright suite, and live failures re-enter the same bounded repair loop.

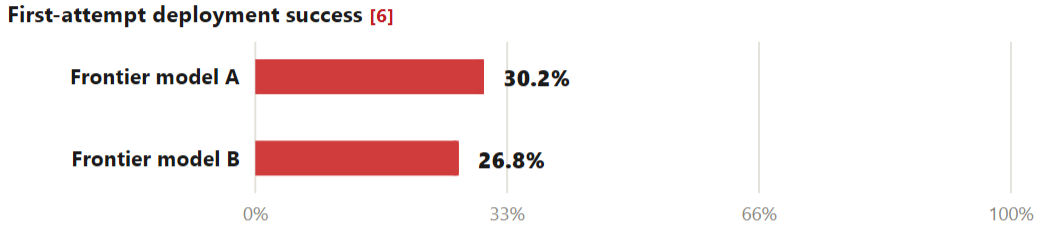
Generation Is Mature — Deployment Success and Human Control Are Not

Agentic generation & issue resolution — a solved-enough problem

LLM4SE survey [1]	926 studies systemised across **112 SE tasks and 5 lifecycle phases** — adoption across the lifecycle is no longer in question.
Issue-resolution survey [12]	150+ primary studies categorised into benchmarks, techniques and empirical work.
Agentless [13]	**Structured beats autonomous:** a deterministic 3-phase pipeline matches or beats complex agents at markedly lower budget.
RepairAgent [14]	Autonomous LLM agent that plans and invokes repair tools — competitive real-world repair rates.
Huang et al., TSE [15]	Repair quality is highly sensitive to **how the task is framed and what context is supplied**.

Design consequence: ForgeFlow takes the **Agentless** lesson, not the autonomous-agent one — a fixed deterministic pipeline, with a 44-file skeleton *copied* rather than generated.

But the deployed artifact still fails



And developers will not cede control

Misalignment at scale [7]	**20,574 real-world agent sessions** show pervasive developer–agent misalignment — actions diverge from intent.
Professional practice [8]	Experienced developers don't "vibe" — they constantly restrain, monitor and redirect agents.
HULA [21][22]	Deployed in Atlassian Jira: engineers keep control at every step. Open challenges named: the **cost of correctness testing** and LLM-evaluation variability.
Design2Code [23]	Multimodal design→code still lags on visual recall and layout fidelity — so design stays **pluggable and human-refinable**.

> The field converged on generation. It did not converge on **deploying successfully, or on letting the human stay in charge**.

Repair Has Diminishing Returns — and Generated Tests Decay When Code Changes

The repair loop has an economics

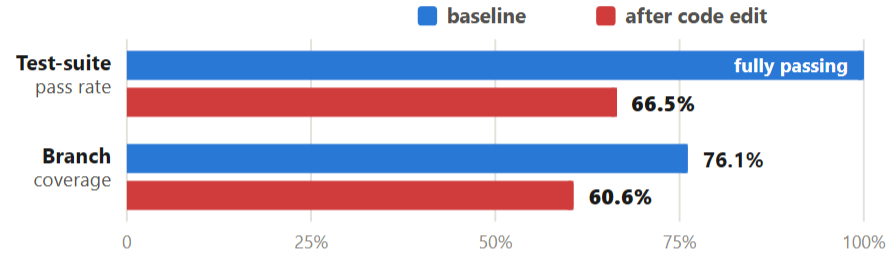
> *The vast majority of the benefit of a repair loop is gained in the* **first one to three iterations** *— later iterations mostly consume tokens for little further gain.*
>
> Kiecker et al. [9] · Arimbur [10]

Security cost [5]	Iterating self-improvement is not merely low-yield past a point — repeated "make it more secure" rounds can **monotonically weaken** security.
Build-code quality [4]	A 2026 mining study finds agents systematically introduce maintainability and security smells into generated build code.
Traceability [20]	TraceLLM *recovers* requirement-to-artifact links after the fact; ForgeFlow builds the link in **by construction** via `criterion_id`.

Design consequence: an unbounded repair loop is a cost sink *and* a safety risk. Our iteration cap (default 5, just above the 1–3 knee), regression guard, stall guard and budget guard are a **response to this literature**.

The strongest 2026 evidence — LLM tests under software evolution [11]

8 LLMs · **22,374 program variants** · **≈346M tokens**



>99% of the failing tests **pass on the original program** — "residual alignment": the model tests the code it *remembers*, not the code it was *given*.

> *"Computing a code diff and explicitly highlighting the modified regions could help guide the model's attention."*
>
> **— the authors' recommended fix** [11]

This is **precisely ForgeFlow's diff-aware minimal repair context** — designed and built, then independently corroborated by this study.

> A March-2026 study, independently, tells you to do **the exact thing we built** — and to stop early.

RESEARCH GAPS (3/3) — CAPABILITY COMPARISON

No Existing System Verifies, Deploys, Validates Live — and Stops

	Research agents [12]–[14]	LLM test / repair tools [15]–[19]	Commercial builders [2]	ForgeFlow
Multimodal design intake (screenshot → UI)	✗	✗	✓	✓ pluggable
Full-stack app generation	◑	✗	✓	✓ fixed stack
Requirements → executable tests (traceable)	✗	◑	✗	✓ structured
Bounded repair — iteration cap + stall detection	✗	✗	✗	✓
Diff-aware minimal repair context	✗	◑	✗	✓
Human escalation on non-convergence	✗	✗	✗	✓
Automated cloud deployment	✗	✗	✓ locked-in	✓ dual-mode
Post-deployment live validation	✗	✗	✗	✓
Non-linear, skip-any-stage control	✗	✗	◑	✓
Per-run cost accounting + budget caps	✗	✗	✗	✓

✓ supported ◑ partial ✗ absent — highlighted rows: where the whole field says no

Research systems verify but do not deploy. Industry systems deploy but do not verify. **Neither validates the live artifact — and none bound the repair loop.**

PROBLEM-TO-DESIGN MAPPING AS PER THE GAP IDENTIFIED (1/2)

Every Identified Gap Maps to a Specific, Built Design Decision

> Current AI app-builders can generate and deploy web applications but cannot *reliably verify* them — leaving users in unbounded, costly bug-fix loops, with no validation of the live deployment.

IDENTIFIED GAP (RESEARCH GAPS §)	FORGEFLOW DESIGN PRINCIPLE	WHERE IT LIVES
AI-generated code is systematically flawed in maintainability & security [4][5]	**P1 · Verification over generation** — knowing when the system is finished, not how fast it emits code	Requirements → executable-test compilation (Build/Test stages)
Self-refinement can **weaken** security; gains plateau after 1–3 iterations [5][9][10]	**P6 · Bounded, non-linear repair** — iteration cap, regression guard, stall guard, budget guard	Self-healing repair loop **— core contribution**
First-attempt infrastructure deployment succeeds only 26.8–30.2% of the time [6]	**Infra-aware, dual-mode deploy** — classify needs before deploying; re-validate the live artifact after	Deploy + Validate stages
Generated tests silently lose validity as code evolves; >99% of new failures still pass on the original program [11]	**Diff-aware minimal repair context** — failing tests + referenced sources + git diff since last green + criterion text only	Repair-context assembly (Section V equivalent)
Developers do not cede control — misalignment appears exactly when they can't intervene [7][8]	**P4 · Never overwrite, always version** — every stage optional, refinable, re-enterable; conductor owns transition legality	Non-linear stage machine (Requirements→Design→Build→Test→Deploy→Validate)
Elaborate autonomous agency underperforms structured pipelines at higher cost [13]	**P2 · One fixed generated-app stack** — hand-written skeleton copied, not generated	44-file app-skeleton template

> The negative commitment, not the positive one: ForgeFlow's differentiator is **knowing when to stop** — every gap above maps to a guard, a boundary, or a versioned artifact.

Requirements-to-Design Mapping — Every Requirement Has a Named Design Element

ID	REQUIREMENT	DESIGN ELEMENT THAT REALISES IT	WHERE IT IS IMPLEMENTED
FR-1	Accept an app idea as **screenshots or a structured spec**, and let the design stage be refined or skipped	Pluggable **DesignProvider** interface (Stitch MCP default, Figma alt); every design output is a new versioned artifact	`design/providers/*`
FR-2	Capture requirements in a form that is **deterministically evaluable**, not free prose	**RequirementSpec** — features → inputs, expected behaviours, acceptance criteria; guided entry + never-auto-saved AI draft	`projects/requirements`
FR-3	Compile requirements into **executable tests traceable back to the criterion**	Test-gen agent emits Vitest / Jest+supertest / Playwright tests, each tagged with the originating `criterion_id`	`agents/testgen · testing/`
FR-4	Generate a working **full-stack application** with high first-pass success	**P2 — one fixed stack**; a hand-written 44-file skeleton is *copied*, not generated, removing all scaffolding tokens	`templates/app-skeleton`
FR-5	Repair test failures **without thrashing or burning budget**	**Bounded repair controller** — regression → stall → iteration cap (5) → budget guard, evaluated in that precedence order	`testing/repair_loop`
FR-6	Focus the repair model on **what actually changed**	**Diff-aware minimal context**: failing tests + referenced sources + git diff since last *green* run + criterion text; hard 24k-char cap	`testing/repair_context`
FR-7	Treat **non-convergence as a normal outcome**, never a silent failure	**Escalation as a terminal state** — stage → `awaiting_user`, `repair.escalation` event with failed tests, diffs tried, metrics, resume contract	`testing/escalation`
FR-8	Deploy correctly **without the user understanding cloud infrastructure**	**Infra analyzer** → `infra_plan` artifact classifying the app and naming exact env vars/secrets; dual-mode (Seamless / BYO) credentials	`deploy/analyzer`
FR-9	**Prove the deployed artifact works** — not just the sandbox copy	Same Playwright suite re-pointed at the live URL; identical normalised result shape, so live failures re-enter the same repair loop	`deploy/validate`
NFR-1	**No stage may be mandatory**; refinement must never destroy work	**StageState** per stage {empty, in_progress, awaiting_user, complete, skipped, stale} + **Conductor** owns transition legality; staleness cascades, never deletes	`orchestrator/conductor`
NFR-2	Untrusted generated code must **never execute in the control plane**	Per-project Docker sandbox — non-root, no host network, dropped capabilities, CPU/memory/PID caps, idle reaping	`sandbox/manager`
NFR-3	**Per-project data isolation** and no credential leakage	One MongoDB database per project + `forgeflow_meta`; provider tokens Fernet-encrypted at rest, decrypted only at call time	`db/ · deploy/vault`
NFR-4	Agent work must be **cost-bounded and attributable**	Two-tier model routing (Sonnet synthesis / Haiku routing) + per-run token & ₹ accounting against per-project and global budget caps	`agents/router · core/cost`

Thirteen requirements, thirteen named design elements — all implemented. The mapping is traceable in both directions via the `criterion_id` chain.

Three Planes and a Provider Boundary — Untrusted Code Is Fenced In

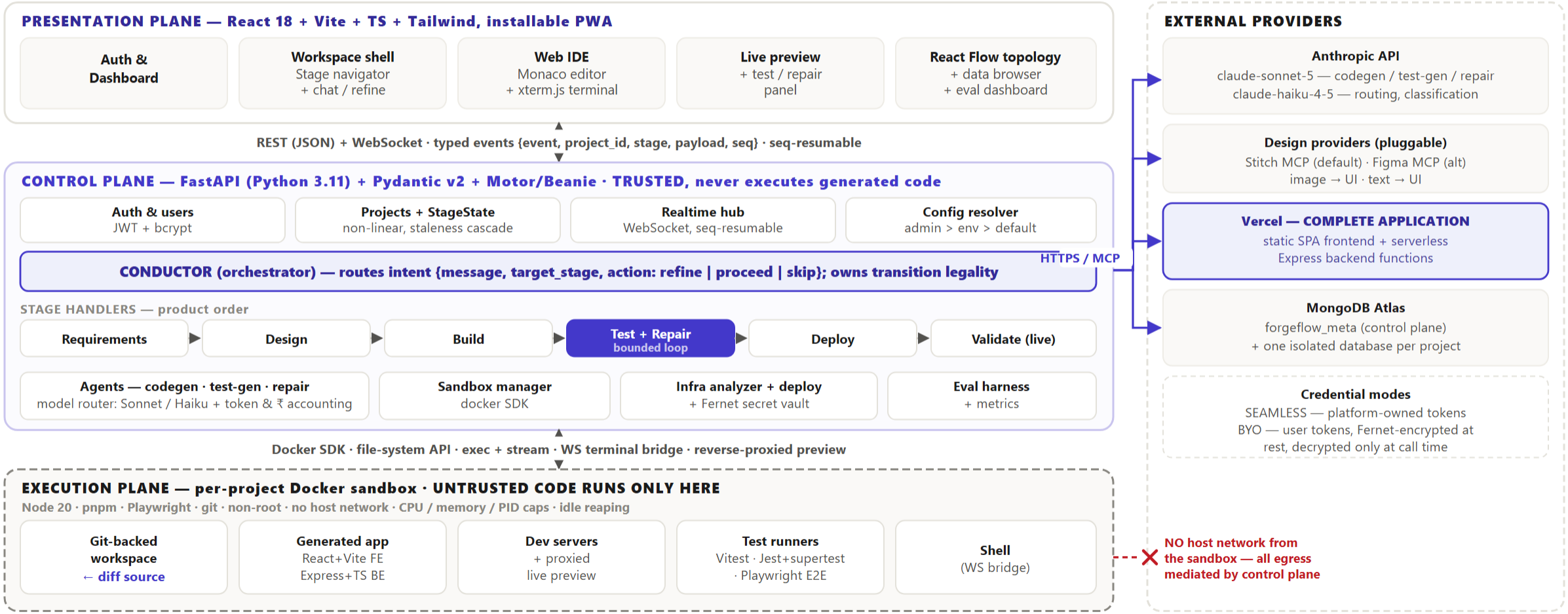


Fig. 1 — ForgeFlow system architecture. Every component shown is implemented; the dashed boundary is the security control [24].

Database Connectivity — One Meta Store, One Isolated Database Per Project



Driver path
Control plane → Motor (async) + Beanie ODM. Generated apps → Mongoose. Two ODMs because the planes are different languages against one cluster.

Credential path
Connection strings are Fernet-encrypted at rest, decrypted only at call time, and injected as an env variable — never logged, never sent to the client.

Ordered deployment
Database provisioned → backend deployed with the DB URI injected → frontend deployed with the API URL injected → every node health-checked.

> **Deployment change since Review 1:** the backend previously ran on Render as a separate persistent process. It is now a **Vercel serverless function alongside the frontend** — one provider, one ordered deploy, one less cross-provider failure mode [6].

Justification for Each Tool — the ForgeFlow Platform Itself

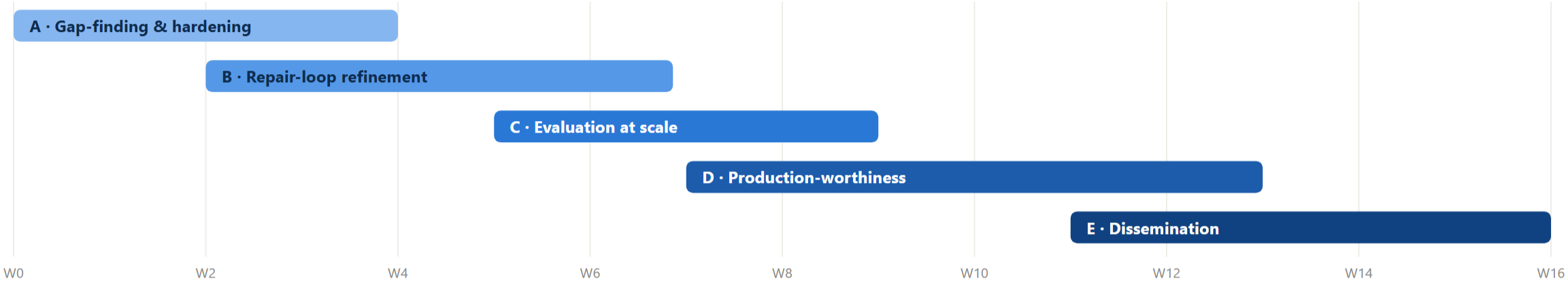
LAYER	TOOL CHOSEN	WHY THIS TOOL	CONSIDERED & REJECTED
API framework	FastAPI (Python 3.11)	Native `async` I/O suits long-running agent, Docker and provider calls; first-class WebSocket support for the resumable event stream; auto-generated OpenAPI keeps the typed client honest.	Django/Flask — sync-first, heavier for a WS-driven API
Validation	Pydantic v2	Schemas at every boundary reject malformed **LLM output before it reaches the sandbox**; v2's Rust core keeps per-request validation cost negligible.	dataclasses / marshmallow — no coercion or JSON-schema export
Database	MongoDB	Artifacts are versioned, heterogeneous and schema-drifting by design (designs, diffs, test runs, repair attempts). A document store absorbs that without migrations, and one database per project gives isolation for free.	PostgreSQL — rigid schema, migration churn per artifact type
DB driver / ODM	Motor + Beanie	Motor is the async driver that matches FastAPI's event loop (a sync driver would block it); Beanie layers typed Pydantic documents on top, so DB models and API models share one definition.	PyMongo — blocking; raw Motor — untyped documents
LLM provider	Anthropic API — `claude-sonnet-5` + `claude-haiku-4-5`	**Two-tier routing is a cost-control decision:** Sonnet only for real synthesis (codegen, test-gen, repair), Haiku for classification, summaries and the freeform-to-feature-list draft. Every call is costed against a budget cap [22].	Single-model routing — 3–5× cost for identical routing quality
Realtime	WebSockets (typed, `seq`-numbered events)	Generation and repair run for minutes; clients must survive a page reload. Monotonic `seq` lets a client resume exactly where it disconnected instead of replaying or losing a run.	SSE — one-directional; polling — misses in-flight loop state
Frontend	React 18 + Vite + TypeScript + Tailwind	Vite's HMR suits an IDE-like workspace; `strict` TS mirrors the generated-app stack so one mental model covers both; Tailwind semantic tokens give light/dark without a parallel stylesheet.	Next.js — SSR is dead weight for an authenticated SPA
Client state	Zustand + TanStack Query	Zustand holds ephemeral UI/stage state; TanStack Query owns server cache, retries and invalidation — so stage transitions refetch correctly without hand-rolled polling.	Redux — boilerplate disproportionate to the state size
Embedded IDE	Monaco Editor + xterm.js	The exact editor and terminal engines VS Code ships. Generated code stays **inspectable and editable rather than a black box** — the directly-addressed trust failure [7][8].	CodeMirror — weaker TS language services
Deploy topology UI	@xyflow/react (React Flow)	The deployment plan is a graph (DB → backend → frontend); rendering it as an interactive node graph with per-node status, logs and redeploy controls makes infrastructure legible to a non-expert.	Static diagram — cannot carry live node status

Justification for Each Tool — What ForgeFlow Produces and Where It Runs

LAYER	TOOL CHOSEN	WHY THIS TOOL	CONSIDERED & REJECTED
Generated stack (fixed)	React (Vite+TS) · Express (TS) · MongoDB + Mongoose · Zod	**One fixed target is the enabling constraint.** It raises first-pass generation success, cuts repair iterations, and lets a hand-written 44-file skeleton be *copied instead of generated* — removing scaffolding tokens entirely (the Agentless lesson [13]).	Multi-stack support — makes rigorous verification intractable
Unit tests (FE)	Vitest + Testing Library	Shares Vite's transform pipeline, so generated tests run under the same config as the app — no second build to keep in sync. Behaviour-level queries survive the cosmetic DOM churn typical of regenerated UI.	Jest for FE — separate transform config to maintain
Unit tests (BE)	Jest + supertest	supertest exercises the Express app in-process, so backend acceptance criteria are testable without provisioning a port or a live server inside the sandbox.	Manual HTTP harness — flaky, port-bound
E2E tests	Playwright	**The same suite runs twice** — against the sandbox preview during Test, then re-pointed at the deployed URL during Validate. One framework, identical normalised results, so live failures re-enter the same repair loop [18].	Cypress — weaker multi-origin & headless-CI story
Execution sandbox	Docker — Node 20, pnpm, Playwright, git, non-root	The security control is **the boundary, not the model's good behaviour** [24]: no host network, dropped capabilities, CPU/memory/PID caps, idle reaping. pnpm's content-addressed store keeps repeated installs cheap across projects.	Direct host exec — unacceptable; VMs — too slow to reap per project
Version control	git (inside the sandbox workspace)	**Not incidental — it is what makes the core contribution computable.** The repair context needs the diff *since the last passing run*; a Git-backed workspace with a moving green reference is what produces it [11].	Snapshot copies — no cheap, semantic line-level diff
Design intake	Stitch MCP (default) / Figma MCP (alt)	A pluggable provider keeps design a **human-refinable, versioned stage** rather than a one-shot conversion — multimodal design-to-code still lags on visual recall and layout fidelity [23]. Quota exhaustion fails soft with a fallback.	Hard-coded single provider — no fallback on quota loss
Hosting	Vercel — **complete application** (static SPA + serverless Express functions)	**Changed since Review 1.** Hosting both tiers on one provider removes an entire ordered cross-provider deploy step and its failure mode — the exact class of failure behind the 26.8–30.2% first-attempt deployment rate [6]. Env vars carry `MONGODB_URI` at build & runtime.	~~Vercel + Render split~~ — two providers, two failure surfaces
Secrets	`cryptography` — Fernet	Authenticated symmetric encryption for BYO provider tokens: encrypted at rest, decrypted only in-process at call time, never logged, never returned to the client. Write-only at the API surface.	Plaintext env storage — unacceptable for user tokens
Quality gates	ruff + black + mypy · ESLint + Prettier · pytest	The control plane is `mypy`-clean with full type hints and no bare excepts; a platform that claims to verify generated code must hold itself to a stricter bar than the code it emits.	Lint-only CI — misses type regressions at agent boundaries

PLAN OF NEXT WORK

Sixteen Weeks — Prove It, Break It, Harden It



Gap-finding & hardening · W1–4

- Failure-mode audit: adversarial specs, ambiguous requirements, sandbox edge cases
- Sandbox security review — isolation, caps, secrets
- Resilience: provider fallbacks, crash recovery, resumable tasks

Repair-loop refinement · W3–7

- Tune iteration caps & stall thresholds against measured convergence
- Regression-guard on every patch
- **Three-arm ablation: no-repair vs bounded diff-aware repair vs diff-context-ablated**

Evaluation at scale · W6–10

- Expand benchmark from 10 to 20–30 diverse app specifications
- Repeated runs per spec to report variance under model non-determinism
- Report convergence, cost per success, latency + variance

Production-worthiness · W8–14

- Observability: traces, cost dashboards, budget enforcement
- Multi-user robustness, quotas, rate limiting
- PWA finalisation + accessibility; seeded demo + fallbacks

Dissemination · W12–16

- Write-up for a software-engineering venue — the bounded loop + ablation is the paper
- Hackathon / expo submissions; demo runbook
- Open-source the eval harness + benchmark specs

> “ The build — including deploy & live-validate testing — is done. The next five months are about **proving it, breaking it, and hardening it** at scale.

CONCLUSION

Verification, Not Generation, Is the Frontier — and It Is Built

- The industry has an **adoption–trust inversion**: adoption fell 84%→76% year on year [3] while trust in AI accuracy fell to 29% [3] — and 43% of AI code still fails in production after QA [25].
- ForgeFlow addresses the failure that matters — **verification, not generation** — with a bounded, diff-aware, human-escalating repair loop.
- A **complete first prototype is implemented**: 53 phases, 12 epics, ≈41,000 lines of production code and 1,491 automated tests — screenshots → deployed and re-validated URL.
- The design is **independently corroborated**: a March-2026 study of 22,374 program variants recommends diff-highlighted repair context after finding >99% residual alignment in LLM-generated tests [11].
- The differentiator is defensible: research verifies without deploying; industry deploys without verifying. **ForgeFlow closes both loops — and stops.**
- **New this cycle:** deploy → live-validate testing is complete, and the platform now hosts the full generated application — frontend and backend — on a single provider (Vercel), removing the ordered cross-provider deployment failure mode identified in [6].
- **Next:** the three-arm evaluation (Section VIII of the paper), adversarial gap-finding, and production-readiness hardening.

> **Anyone can make an AI write an app. The hard part is knowing when it's actually *finished* — and that is what we built.**

References

[1] Q. Zhang *et al.*, "A survey on large language models for software engineering," *Sci. China Inf. Sci.*, vol. 69, no. 4, Art. no. 141102, Mar. 2026, doi: 10.1007/s11432-025-4670-0.

[2] Y. Ge *et al.*, "A survey of vibe coding with large language models," arXiv:2510.12399, Oct. 2025.

[3] Stack Overflow, "2025 Developer Survey," Stack Overflow, 2025. [Online]. Available: https://survey.stackoverflow.co/2025/

[4] A. Ghammam and M. Almukhtar, "AI builds, we analyze: An empirical study of AI-generated build code quality," in *Proc. 23rd Int. Conf. Mining Softw. Repositories (MSR)*, Rio de Janeiro, Brazil, 2026, pp. 727–731, doi: 10.1145/3793302.3793563.

[5] S. Shukla, H. Joshi, and R. Syed, "Security degradation in iterative AI code generation: A systematic analysis of the paradox," in *Proc. 2025 IEEE Int. Symp. Technol. Soc. (ISTAS)*, Santa Clara, CA, USA, 2025, pp. 1–8, doi: 10.1109/ISTAS65609.2025.11269659.

[6] T. Zhang, S. Pan, Z. Zhang, Z. Xing, and X. Sun, "Deployability-centric infrastructure-as-code generation: An LLM-based iterative framework," arXiv:2506.05623, Jun. 2025.

[7] N. Tang *et al.*, "How coding agents fail their users: A large-scale analysis of developer-agent misalignment in 20,574 real-world sessions," arXiv:2605.29442, May 2026.

[8] R. Huang, A. Reyna, S. Lerner, H. Xia, and B. Hempel, "Professional software developers don't vibe, they control: AI agent use for coding in 2025," arXiv:2512.14012, Dec. 2025.

[9] T. Kiecker, E. Reichmann, H. Kang, G. An, and L. Grunske, "Is three the magic number? An empirical evaluation of LLM-based repair loops," arXiv:2607.05197, Jul. 2026.

[10] J. J. Arimbur, "How many tries does it take? Iterative self-repair in LLM code generation across model scales and benchmarks," arXiv:2604.10508, Apr. 2026.

[11] S. Haroon, M. T. Khan, and M. A. Gulzar, "Evaluating LLM-based test generation under software evolution," arXiv:2603.23443, Mar. 2026.

[12] Z. Jiang, D. Lo, and Z. Liu, "Agentic software issue resolution with large language models: A survey," arXiv:2512.22256, Dec. 2025.

[13] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 801–824, 2025, doi: 10.1145/3715754.

[14] I. Bouzenia, P. T. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Ottawa, ON, Canada, 2025, pp. 2188–2200, doi: 10.1109/ICSE55347.2025.00157.

[15] K. Huang, J. Zhang, X. Bao, X. Wang, and Y. Liu, "Comprehensive fine-tuning large language models of code for automated program repair," *IEEE Trans. Softw. Eng.*, vol. 51, no. 4, pp. 904–928, Apr. 2025, doi: 10.1109/TSE.2025.3532759.

[16] W. C. Ouédraogo *et al.*, "Prompt engineering in LLMs for automated unit test generation: A large-scale study," *Empirical Softw. Eng.*, vol. 31, no. 4, Art. no. 103, 2026, doi: 10.1007/s10664-026-10840-4.

[17] Y. Shang, Q. Zhang, C. Fang, S. Gu, J. Zhou, and Z. Chen, "A large-scale empirical study on fine-tuning large language models for unit testing," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, Art. no. ISSTA074, pp. 1678–1700, 2025, doi: 10.1145/3728951.

[18] P. Alian, N. Nashid, M. Shahbandeh, T. Shabani, and A. Mesbah, "Feature-driven end-to-end test generation," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng. (ICSE)*, Ottawa, ON, Canada, 2025, pp. 450–462, doi: 10.1109/ICSE55347.2025.00141.

[19] S. Gu, N. Nashid, and A. Mesbah, "LLM test generation via iterative hybrid program analysis," arXiv:2503.13580, Mar. 2025.

[20] N. Alturayeif, I. Ahmad, and J. Hassine, "TraceLLM: Leveraging large language models with prompt engineering for enhanced requirements traceability," *Requirements Eng.*, vol. 31, no. 1, Art. no. 6, 2026, doi: 10.1007/s00766-026-00460-1.

[21] W. Takerngsaksiri *et al.*, "Human-in-the-loop software development agents," in *Proc. IEEE/ACM 47th Int. Conf. Softw. Eng.: Softw. Eng. in Practice (ICSE-SEIP)*, Ottawa, ON, Canada, 2025, pp. 342–352, doi: 10.1109/ICSE-SEIP66354.2025.00036.

[22] J. Pasuksmit *et al.*, "Human-in-the-loop software development agents: Challenges and future directions," in *Proc. IEEE/ACM 22nd Int. Conf. Mining Softw. Repositories (MSR)*, Ottawa, ON, Canada, 2025, pp. 756–757, doi: 10.1109/MSR66628.2025.00112.

[23] C. Si, Y. Zhang, R. Li, Z. Yang, R. Liu, and D. Yang, "Design2Code: Benchmarking multimodal code generation for automated front-end engineering," in *Proc. 2025 Conf. Nations Americas Chapter Assoc. Comput. Linguist.: Human Lang. Technol. (NAACL)*, Albuquerque, NM, USA, 2025, pp. 3956–3974, doi: 10.18653/v1/2025.naacl-long.199.

[24] R. Rabin, J. Hostetler, S. McGregor, B. Weir, and J. Judd, "SandboxEval: Towards securing test environment for untrusted code," arXiv:2504.00018, Mar. 2025.

[25] Lightrun, "2026 State of AI-Powered Engineering Report," Lightrun Ltd., 2026. [Online]. Available: https://venturebeat.com/technology/43-of-ai-generated-code-changes-need-debugging-in-production-survey-finds

[26] GitClear, "The Maintainability Gap: 2026 AI Code Quality Research," GitClear, 2026. [Online]. Available: https://www.gitclear.com/

[27] Veracode, "2025 GenAI Code Security Report," Veracode Inc., 2025. [Online]. Available: https://www.veracode.com/

[28] CodeRabbit, "AI-Generated Code Quality: An Analysis of 470 Real-World Pull Requests," CodeRabbit, 2026. [Online]. Available: https://byteiota.com/ai-code-quality-crisis-84-adoption-29-trust-in-2026/

[29] Hostinger, "Vibe Coding Statistics 2026: Adoption, Productivity, and Security Data," 2026, citing Gartner and Forrester forecasts. [Online]. Available: https://www.hostinger.com/blog/vibe-coding-statistics/